

24주 차 보고서

(2026년 6월 8일 ~ 2026년 6월 13 일)

작성자

박신우

이번 주 회의 내용

이번 주 한 일

06.12 2스테이지 맵 윤진이가 만든 타일 적용

윤진이가 만든 2스테이지 타일맵으로 무한맵이 되도록 하려 한다.

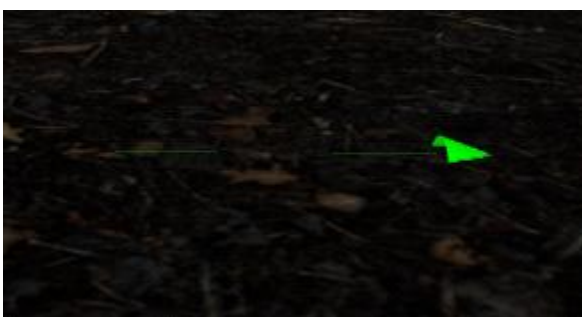


맵이 로딩이 되질 않길래 찾아보니 타일 마커를 맵에다가 안 둔 상태였고, entry point를 맵의 시작지점에 배치 한 후 실행을 해보았더니,

맵 한 가운데에서 트럭과 플레이어가 스폰이 되었다.



분명 entry point에서 스폰이 되어 시작이 되도록 하였는데 싫어서, 타일맵을 확인해 보니 다음과 같이 entry point가 0,0,0 지점으로 이동이 되어있었다.



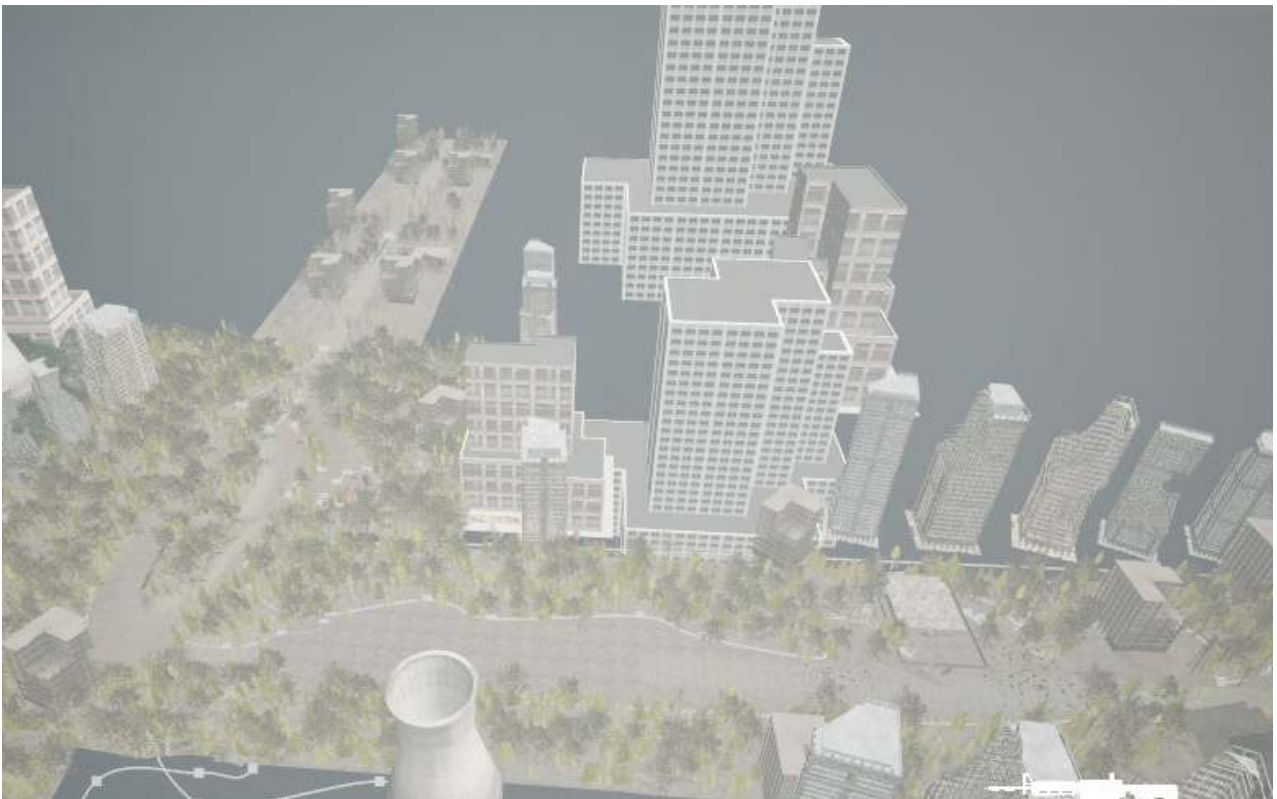
Transform			
Location	0.0	0.0	0.0
Rotation	-0.0°	0.0°	0.0°
Scale	1.0	1.0	1.0

이번 주 한 일

뭔가 이상하다 싶어서 코드를 찾아보니, 에디터에서 배치하거나 변경할 때, 위치와 회전값을 0으로 설정하게 해놨었다.
굳이 저 기능이 필요없다 싶어서, 주석처리 해두고 다시 실행하였다.

```
// 게임중에 호출되지 않음, 에디터에서 배치하거나 변경이 될 때 호출이 됨.  
void AStage2TileMarker::OnConstruction(const FTransform& Transform)  
{  
    Super::OnConstruction(Transform);  
  
    // 시작 화살표 설정  
    if (EntryArrow)  
    {  
        EntryArrow->SetRelativeLocation(FVector::ZeroVector);  
        EntryArrow->SetRelativeRotation(FRotator::ZeroRotator);  
    }  
}
```

다시 실행하니 정상적으로 시작이 되었고,
맵의 위치를 좀 조절한 후 잘 이어지는 모습을 확인 할 수 있었다.



06.13 2스테이지 풀에 있는 타일맵 렌더링 하지 않도록

```

void AStage2TileManager::SetTileRenderingEnabled(const FStage2LoadedTile& LoadedTile, bool bEnabled) const
{
    if (!bHidePooledTiles || !LoadedTile.StreamingLevel)
    {
        return;
    }

    ULevel* LoadedLevel = LoadedTile.StreamingLevel->GetLoadedLevel();
    if (!LoadedLevel)
    {
        return;
    }

    // 모델 컴포넌트 숨기거나 안숨겨지게 함
    for (ULevelComponent* ModelComponent : LoadedLevel->ModelComponents)
    {
        if (ModelComponent)
        {
            ModelComponent->SetHiddenInGame(!bEnabled);
        }
    }

    // 액터 컴포넌트 숨기거나 안숨겨지게 함
    for (AActor* LevelActor : LoadedLevel->Actors)
    {
        if (IsValid(LevelActor))
        {
            LevelActor->SetActorHiddenInGame(!bEnabled);
        }
    }
}

```

2스테이지 무한맵 구현을 위해 타일맵을 미리 풀링해두었는데,
 풀에 들어간 타일들이 사용되지 않는 상태에서도 렌더링 대상으로 남아 있었다.
 이로 인해 카메라가 특정 방향을 바라볼 때 사용 중인 타일뿐 아니라 대기 중인 풀 타일들
 까지 함께 렌더링되었고,
 스켈레탈 메시 렌더링에 사용되는 Skin Cache 메모리가 초과되어 OUT OF MEMORY
 FOR SKIN CACHE 오류가 발생했다

OUT OF MEMORY FOR SKIN CACHE, REQUIRES 8.312 extra MB (currently at 128.000)

그래서 풀에 있는 타일 맵들이 렌더링을 하지 않도록 해야겠다고 생각이 들었고,

이번 주 한 일

HiddenInGame 이라는 함수가 있다는 것을 찾아서,
그 함수를 이용하여 타일맵의 Rendering을 끄는 함수를 만들었다.

```
void AStage2TileManager::SetTileRenderingEnabled(const FStage2LoadedTile& LoadedTile, bool bEnabled) const
{
    if (!bHidePooledTiles || !LoadedTile.StreamingLevel)
    {
        return;
    }

    ULevel* LoadedLevel = LoadedTile.StreamingLevel->GetLoadedLevel();
    if (!LoadedLevel)
    {
        return;
    }

    // 모델 컴포넌트 숨기거나 안숨겨지게 함
    for (UModelComponent* ModelComponent : LoadedLevel->ModelComponents)
    {
        if (ModelComponent)
        {
            ModelComponent->SetHiddenInGame(!bEnabled);
        }
    }

    // 액터 컴포넌트 숨기거나 안숨겨지게 함
    for (AActor* LevelActor : LoadedLevel->Actors)
    {
        if (IsValid(LevelActor))
        {
            LevelActor->SetActorHiddenInGame(!bEnabled);
        }
    }
}
```

저 함수를 이용하여 풀에 있거나 풀로 돌아갈 때 Rendering을 끄도록 만들었고
이로 인하여, 대기 중인 타일이 불필요하게 화면에 그려지지 않게 되어 Skin Cache 메모리 사용량이 줄었고, 해당 메모리 부족 오류가 발생하지 않게 되었다

다음 주 할 일

무장상태 캐릭터 애니메이션

특이사항
무장상태 캐릭터 애니메이션이 원래 이번주 목표였으나, 시간상 이유로 다음주로 미루었습니다.

24주 차 보고서

(2026년 6 월 7 일 ~ 2026년 6 월 13 일)

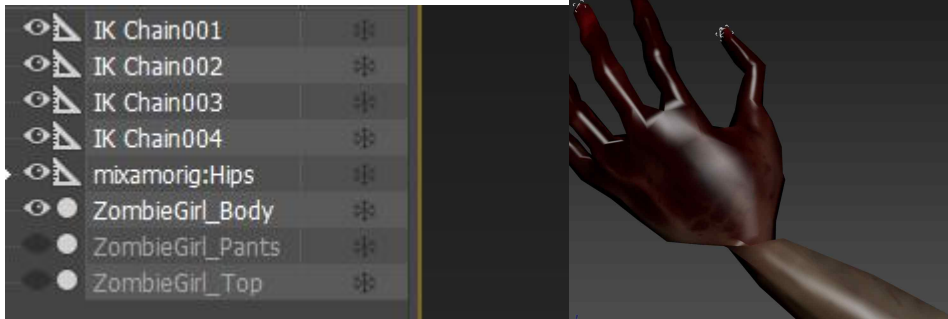
작성자

안윤진

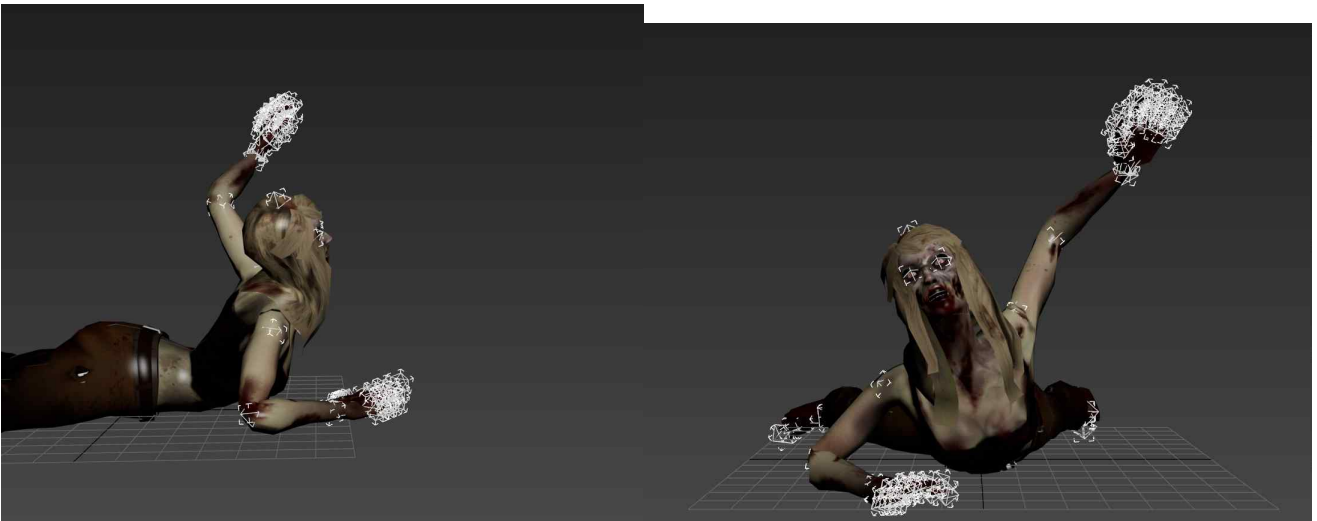
이번 주 한 일



T포즈에서 일단 얹어놓는 것부터 시작한다.



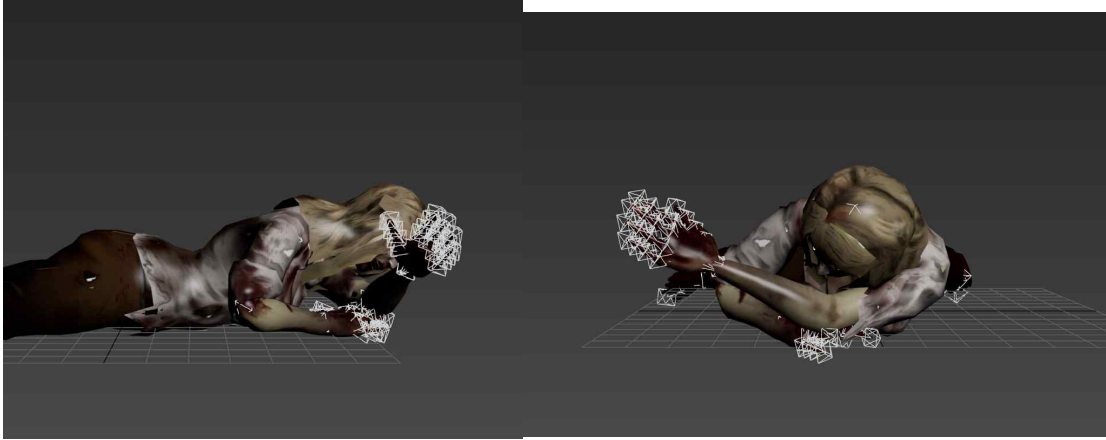
이렇게 관절을 막 휘다보면 몸 메쉬가 깨지는 걸 볼 수 있다.. 하나씩 잘 확인하면서 회전시켜준다.
뼈들도 연결시켜 관절같이 제작해보려 했으나, 실행에서 생각 외의 변수가 많아 다 지웠다.



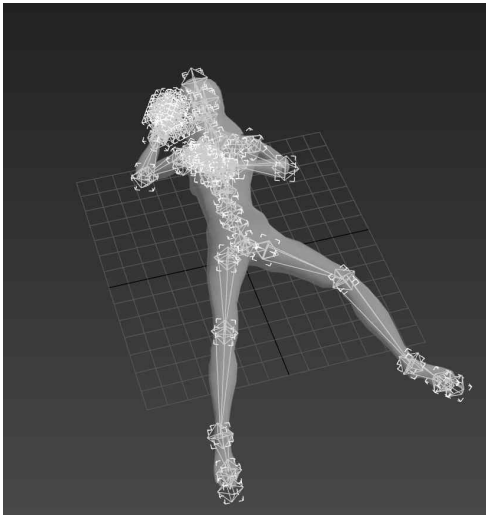
키 프레임이 될 장면들을 먼저 잡아준다. 핵심키프레임1공격 (팔들기)

핵심 키프레임 틀은 1->2->1 로 구성. 그 후 중간중간에서의 디테일을 더 잡아준다.

이번 주 한 일



핵심 키프레임2(팔 휘두른 뒤)



너무 뭉쳐있어서 특히 손 부분이 보기 너무 힘들다. 기존 바이패드를 사용한 애니메이션만 주로 배웠는데, 이렇게 전부 따로 조정된 스켈레톤 메쉬를 다루려니 원하는 바가 잘 안나온다. 투박하게 나온다. 수동으로 하나하나 다 수정해준다..

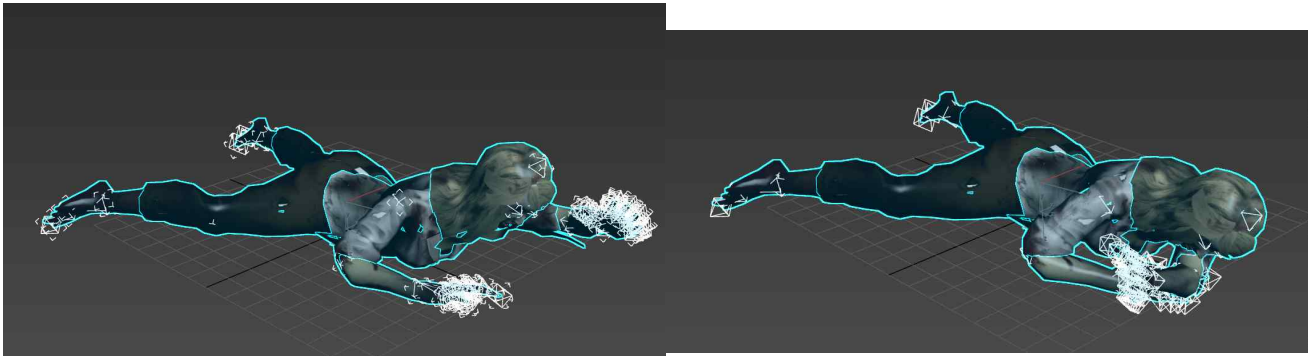
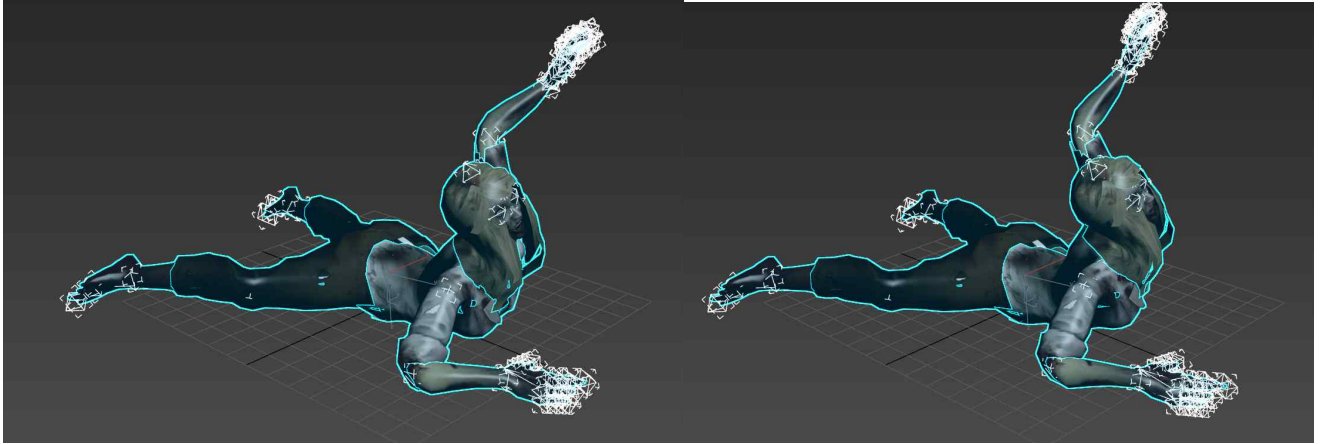


최종적으로 55프레임에 엮어진상태의 공격 모션 구현했다.

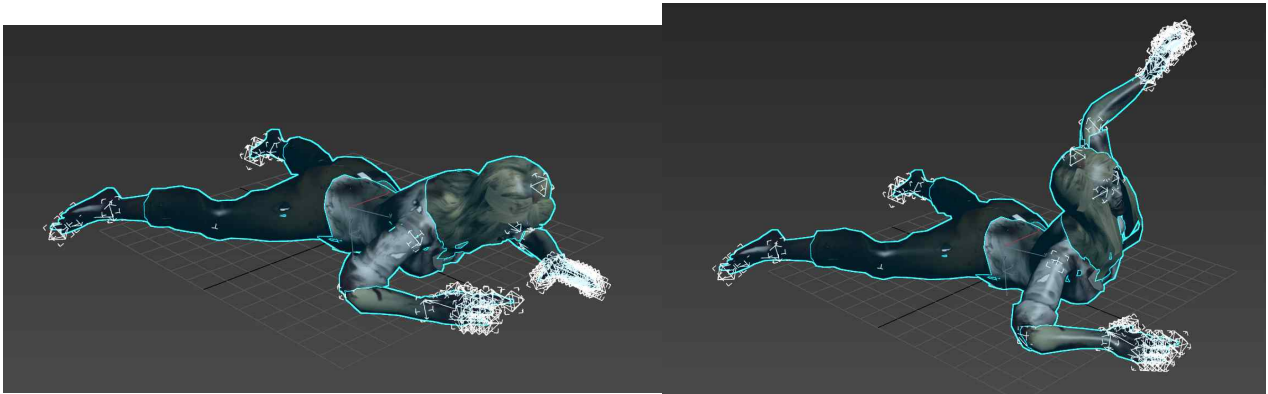


실제 프로젝트에도 잘 적용된 모습이다. export할 때 animation only에 체크했더니 처음에 적용되지 않았다. 그냥 처음 그대로 fbx로 내보내면 된다.

이번 주 한 일



중간중간 몸을 돌릴때마다 오른쪽 팔도 같이 돌아가는데, 해당 팔만 위치에 고정할 줄 몰라서 그나마 이어짐이 자연스럽게 수정하고 오른쪽 팔만 액션기로 고정해줬다.
(들고 -> 더 들고 -> 휘두름) 공격 동작 끝.



하지만 자연스럽게 이어져야하기에 첫 번째 동작으로 돌아가는 과정의 키액션도 제작해준다.
휘두른 후 원위치로 돌아가는 동작. 그 후 다시 첫 번째 동작으로. 여기까지가 55프레임이다.
둘 사이의 텅을 20초 넘게 주었더니 자연스럽게 이어졌다.

다음 주 할 일

특이사항
다음 주는 특히 시험기간으로 못 할 것 같다.

24주 차 보고서

(2026년 6 월 7 일 ~ 2026년 6 월 13 일)

작성자

배주환

이번 주 회의 내용

이번 주 한 일

좀비가 다리가 잘렸을 때 엎드려진 상태에서 다시 공격을 하려고 일어나는 버그가 있어서 수정했다. 다리가 잘린 상태에서는 공격을 하지 않고, 울부짖는 애니메이션만 넣었다.

그리고 다리 절단 후 엎드린 상태에서 공중에 떠 있는 버그가 있어서 수정했다.

엎드린 상태로 전환될 때, StartCrawling()에서 메시의 상대 위치 z값을 강제로

-CrawlingCapsuleHalfHeight로 설정하고 있어서 문제가 되는 것이었다. CrawlingCapsuleHalfHeight의 값은 30.0f였고, 기존에 클라에서 만든 좀비 메시의 기본 z 오프셋보다 훨씬 높은 위치여서 공중에 떠있었던 것이었다.

그래서 BeginPlay()에서 블루프린트가 가진 기본 메시 상대 위치를 저장하도록 하였다.

```
void ABaseZombie::BeginPlay()
{
    Super::BeginPlay();

    if (ZombieMesh)
    {
        StandingMeshRelativeLocation = ZombieMesh->GetRelativeLocation();
        bHasStandingMeshRelativeLocation = true;
    }
}
```

그리고 StartCrawling()에서는 메시 z값을 캡슐 높이에 맞춰 강제로 바꾸지 안혹, 저장해둔 기본 메시 위치를 유지하도록 수정했다. 그 결과 다리 절단 후 공중에 떠 있는 버그가 고쳐졌다.

좀비가 서로 겹치는 문제가 있어서 전에 좀비에게 RVO 회피 범위를 넣어주었는데 실제 좀비 크기보다 과하게 컸던 것 같다. 그래서 좀비가 5마리이상 보이면 강강술래 하듯이 뱅글뱅글 도는 문제가 발생해서 RVO 회피 값을 280->120으로 줄였다.



아직도 5마리 이상이 보이면 뱅글뱅글 도는데 실제 게임을 플레이할때는 플레이어가 멈춰 있는 상황이 거의 없을 것이고, 지금 RVO회피 범위가 적당한 것 같아서 일단은 지금 상태로 유지를 할 것이다.

다음 주 할 일

두번째 스테이지 맵과 좀비의 충돌 구현